UNIVERSITY OF BUEA

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

CEF440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

TASK 1 REPORT: MOBILE APPLICATION DEVELOPMENT PROCESS

GROUP 2

COURSE INSTRUCTOR: Dr. Valery Nkemeni

S/N	NAME	MATRICULE
1.	EFUETNGONG GENESIS TAZISONG	FE23A041
2.	MATUTE MOLUA FRITZ	FE23A087
3.	NWANTOLY STEPHILL FAVOUR BENYELLA	FE23A127
4.	NYANYOH DIVINE-FORTUNE BANFILA	FE23A128
5.	TENKEN MANPLA ULRICH	FE23A170

Contents

1. Introduction 2

2. Types of Mobile Applications 2

Native Applications: 2

Web Applications: 3

Hybrid Applications: 3

Progressive Web Applications (PWAs): 3

3. Mobile App Programming Languages 4

Android (Java, Kotlin): 4

iOS (Swift, Objective-C): 4

Cross-platform (JavaScript, Dart, C#): 4

4. Mobile App Development Frameworks 5

Native Frameworks (Android SDK, iOS SDK): 5

Cross-platform Frameworks (Flutter, React Native, Xamarin): 5

Hybrid Frameworks (Ionic, Apache Cordova): 5

5. Mobile App Architectures and Design Patterns 5

Architectures: 5

Design Patterns: 6

6. Requirement Engineering Process 6

Requirement Gathering (Interviews, Surveys): 6

Requirement Analysis (Functional and Non-functional Requirements): 6

Requirement Specification (SRS Documentation) : 6

Requirement Validation and Management : 6

7. Mobile App Development Cost Estimation 6

Factors Affecting Cost: 7

Estimation Techniques: 7

Cost Categories: 7

8. Conclusion 7

1. Introduction

Mobile application development has become a critical aspect of modern computing, driven by the widespread use of smartphones and tablets. This process involves designing, developing, testing, and deploying software applications tailored for mobile platforms such as Android and iOS

This report explores the mobile application development process by examining the types of mobile applications, programming languages, development frameworks, architectures and design patterns, requirement engineering, and cost estimation. The discussion aligns with the course objectives of understanding mobile development concepts, architectures, user interface design, and deployment strategies.

2. Types of Mobile Applications

Native Applications:

Native applications are mobile applications developed specifically for a particular operating system using platform-specific programming languages, tools, and development environments. For Android, native apps are typically built using Java or Kotlin within android studio, while for iOS, they are developed using Swift or Objective-C within Xcode.

Because native applications are designed for a single platform, they can fully utilize the device's hardware and operating system features such as the camera, GPS, sensors, notifications, and background services. This direct access enables high performance, faster execution, and smoother user experience, making native apps ideal for performance-intensive applications such as gaming, real-time systems, and multimedia applications.

Another key advantage of native applications is their ability to provide a highly optimized and responsive user interface (UI). They follow platform-specific design guidelines such as material design for Android and human interface guidelines for iOS, ensuring consistency and better usability for users.

However, native development has some limitations. Since each platform requires its own codebase, developers must build and maintain separate applications for Android and iOS. This leads to higher development costs, increased time consumption, and the need for specialized expertise in multiple programming languages and tools.

Despite these challenges, native applications remain the preferred choice when performance, security, and user experience are critical, especially in enterprise applications, banking systems, and high-end mobile solutions.

Web Applications:

Web applications are mobile applications that run in a web browser and are developed using HTML, CSS, and JavaScript. They are accessed through a URL and do not require installation, making them easy to use across different devices and platforms.

Their main advantage is cross-platform compatibility, as a single codebase can work on Android, iOS, and other systems, reducing development cost and time. However, web applications have limited access to device features such as sensors and advanced hardware, and they may offer lower performance compared to native applications, especially when handling complex tasks.

Hybrid Applications:

Hybrid applications are mobile apps that combine both web and native features. They are developed using web technologies such as HTML, CSS, and JavaScript, and then wrapped in a native container using frameworks like Ionic or Apache Cordova.

This approach allows developers to use a single codebase across multiple platforms, making development faster and more cost-effective. Hybrid apps can also access some device features like the camera and GPS through plugins.

However, hybrid applications generally have lower performance and responsiveness compared to native apps, especially for complex or high-performance tasks.

Progressive Web Applications (PWAs):

Progressive Web Applications (PWAs) are advanced web applications that provide an app-like experience using standard web technologies such as HTML, CSS, and JavaScript. They can be accessed through a browser but also offer features similar to native apps.

PWAs support offline functionality using service workers, allowing users to continue using the app even without an internet connection. They are also installable on devices without going through app stores, and can send push notifications to users.

Although PWAs are fast and cost-effective, they still have limited access to some device features compared to fully native applications.

3. Mobile App Programming Languages

Mobile application development uses different programming languages depending on the platform.

Android (Java, Kotlin):

Android applications are primarily developed using Java and Kotlin. Java is the traditional language, while Kotlin is now the preferred choice because it is more modern, concise, and reduces coding errors.

iOS (Swift, Objective-C):

iOS applications are built using Swift and Objective-C. Swift is the modern language introduced by Apple, offering better performance, safety, and ease of use, while Objective-C is mainly used for maintaining older applications.

Cross-platform (JavaScript, Dart, C#):

Cross-platform development allows developers to build applications for both Android and iOS using a single codebase.

- JavaScript is used with frameworks like React Native
- Dart is used with Flutter
- C# is used with Xamarin

  This approach reduces development time and cost but may have slight performance limitations compared to native development.

4. Mobile App Development Frameworks

Mobile application frameworks provide tools, libraries, and pre-built components that help developers build applications more efficiently.

Native Frameworks (Android SDK, iOS SDK):

Native frameworks are platform-specific tools used to develop applications for a particular operating system. The Android SDK and iOS SDK provide full access to device features and ensure high performance and seamless user experience.

Cross-platform Frameworks (Flutter, React Native, Xamarin):

Cross-platform frameworks allow developers to build applications for multiple platforms using a single codebase.

- Flutter uses Dart and provides fast performance with a rich UI
- React Native uses JavaScript and reusable components
- Xamarin uses C# and integrates with the .NET ecosystem
  These frameworks reduce development time and cost.

Hybrid Frameworks (Ionic, Apache Cordova):

Hybrid frameworks use web technologies such as HTML, CSS, and JavaScript to build apps that run inside a native container. They are easy to develop and maintain but may have lower performance compared to native apps.

Overall, frameworks improve productivity, reduce development time, and enhance code reusability, making the development process more efficient.

5. Mobile App Architectures and Design Patterns

Mobile application architecture defines how different components of an application are structured and interact with each other, while design patterns provide standard solutions for organizing code effectively.

Architectures:

- Monolithic Architecture.
  In this architecture, all components of the application (UI, business logic, and data handling) are combined into a single unit. It is simple to develop but can be difficult to maintain and scale as the application grows.
- Client-Server Architecture.
  This is the most common architecture in mobile applications, where the mobile app (client) communicates with a remote server to retrieve and store data. It supports scalability, real-time updates, and efficient data management.

Design Patterns:

- MVC (Model-View-Controller)
  Separates the application into three components: Model (data), View (UI), and Controller (logic). This improves organization but can become complex in large applications.
- MVVM (Model-View-View Model):
  Enhances separation by introducing a View Model that manages UI-related data. It supports better data binding and is widely used in modern mobile development.
- MVP (Model-View-Presenter):
  Separates logic into a Presenter, improving testability and making the application easier to maintain.

These architectures and design patterns improve scalability, maintainability, and testability, ensuring that applications are easier to develop, update, and debug.

6. Requirement Engineering Process

The requirement engineering process involves identifying, analyzing, documenting, and managing the needs of users to ensure the successful development of a mobile application.

Requirement Gathering (Interviews, Surveys):

This is the initial stage where information is collected from stakeholders such as users, clients, and developers. Techniques like interviews, questionnaires, and surveys are used to understand user needs and expectations.

Requirement Analysis (Functional and Non-functional Requirements):

In this stage, the collected requirements are examined and categorized.

- Functional requirements define what the application should do (e.g., login, data processing).
- Non-functional requirements define how the system performs (e.g., performance, security, usability).

Requirement Specification (SRS Documentation) :

The requirements are formally documented in a Software Requirement Specification (SRS). This document serves as a reference for developers and ensures clarity and consistency throughout the project.

Requirement Validation and Management :

Requirements are reviewed to ensure they are accurate, complete, and feasible. Requirement management involves handling changes and updates during the development process.

Overall, the requirement engineering process ensures that the final application meets user needs and project objectives.

7. Mobile App Development Cost Estimation

Mobile app development cost estimation is the process of predicting the financial resources required to design, develop, test, and maintain an application. Accurate estimation is essential for effective planning, budgeting, and project success.

Factors Affecting Cost:

Several factors influence the total cost of development:

- Complexity: Simple apps cost less, while complex apps with advanced features (e.g., real-time tracking, AI) are more expensive.
- Platform: Developing for Android, iOS, or both increases cost, especially for native apps.
- Features: Additional functionalities such as authentication, APIs, and UI design increase development effort.
- Team: The size, experience, and location of the development team affect cost.
- Maintenance: Ongoing updates, bug fixes, and improvements add to long-term costs.

Estimation Techniques:

- Expert Judgment: Estimates based on the experience of professionals.
- Analogous Estimation: Based on costs of similar previous projects.
- Parametric Estimation: Uses measurable parameters (e.g., time, resources) to calculate cost.

Cost Categories:

- Development Cost: Coding, UI design, and implementation
- Testing Cost: Quality assurance and debugging
- Deployment Cost: App store publishing and configuration
- Maintenance Cost: Updates, support, and performance improvements

Effective cost estimation helps ensure that projects are completed within budget while maintaining quality.

8. Conclusion

Mobile application development is a comprehensive process that involves multiple stages, including selecting appropriate application types, choosing suitable programming languages and frameworks, designing effective architectures, and applying proper requirement engineering practices. Each stage plays a critical role in ensuring the success of the final product.

A strong understanding of these concepts enables developers to build efficient, scalable, secure, and user-friendly mobile applications. Additionally, the integration of cost estimation, testing, and deployment strategies ensures that applications are developed within budget and meet user expectations.

Overall, mastering the mobile application development process aligns with the course objectives and equips students with the practical and theoretical knowledge required to develop high-quality mobile applications for modern platforms such as Android and iOS.